TASK MANAGEMENT SYSTEM

A Microservices-Based Collaboration Platform
(.NET 10, React, Docker)

Final Project Report

Submitted by:

Denish Murawala
Team Lead — Backend & architecture

Avinash Kanaujia
Frontend & user experience

Anushtup Ganguly
Microservices & integration

October 2026

Abstract

Modern software teams rarely struggle because they lack tools—they struggle because work is scattered across chats, spreadsheets, and half-finished tickets. Our Task Management System addresses that gap with a single web application where projects, sprints, Kanban boards, comments, and file attachments live together. The solution is deliberately built as a set of small .NET microservices behind an Ocelot API gateway, with a React single-page application as the only public client. JWT-based authentication separates administrators from regular users, while Docker Compose makes it practical to run SQL Server, MongoDB, and all services on a developer laptop without cloud subscriptions. This report summarises the motivation, design choices, implementation highlights, and how we validated the system before demonstration.

1. Introduction

The project started from a simple classroom brief: demonstrate real-world .NET skills—not only REST APIs, but also service boundaries, shared security, and a usable front end. We chose task management because every reviewer immediately understands the domain: create a project, plan a sprint, assign work, and track progress on a board. That clarity let us spend more time on architecture than on explaining exotic business rules.

Over the development cycle we organised work around three roles in the team. Denish focused on gateway routing, auth, and cross-service rules (for example ensuring assignees can open tasks even when they are not yet formal project members). Avinash shaped the React UI—Kanban drag-and-drop, forms, and the overall workflow for admins versus everyday users. Anushtup connected services end to end: Docker scripts, health checks, notification hubs, and keeping local ports consistent with Ocelot configuration.

1.1 Objectives

- Deliver a working SPA that talks to one gateway URL in development and production-style setups.
- Split ownership by bounded context: identity, projects/teams/sprints, tasks/subtasks, comments, files, dashboard metrics, notifications, and optional AI chat.
- Enforce role-based access so admins manage structure while users execute day-to-day work safely.
- Provide repeatable local startup (scripts + Compose) and automated tests so regressions are caught early.

2. Requirements and System Design

Functional requirements were captured in plain language first, then mapped to services. Administrators create teams and projects, define sprints, invite members, and assign tasks. Both admins and authorised users create tasks, move cards across statuses (Todo → In Progress → Testing → Done), complete subtasks, comment on work, and upload attachments. The dashboard

aggregates counts for quick situational awareness, and SignalR pushes notification events to the browser.

2.1 Role matrix

Capability	Admin	User (with project access)
Create / delete teams & projects	Yes	No
Manage sprints & team membership	Yes	No
Assign tasks to members	Yes	No
Kanban board & task editing	Yes	Yes
Comments & file uploads	Yes	Yes
Dashboard & notifications	Yes	Yes

2.2 Architecture

Clients never call microservices directly. The React app stores a JWT after login and sends all API traffic to the gateway on port 5204. Ocelot forwards paths such as /api/auth, /api/projects, /api/tasks, and /api/chat to the correct downstream host. Auth issues tokens; Project owns teams, projects, and sprints; Task owns tasks and subtasks; Comment persists threads in MongoDB; File stores binaries locally in development; Notification exposes REST plus hubs; Dashboard reads aggregated data; AI proxies prompts to Gemini when an API key is configured.

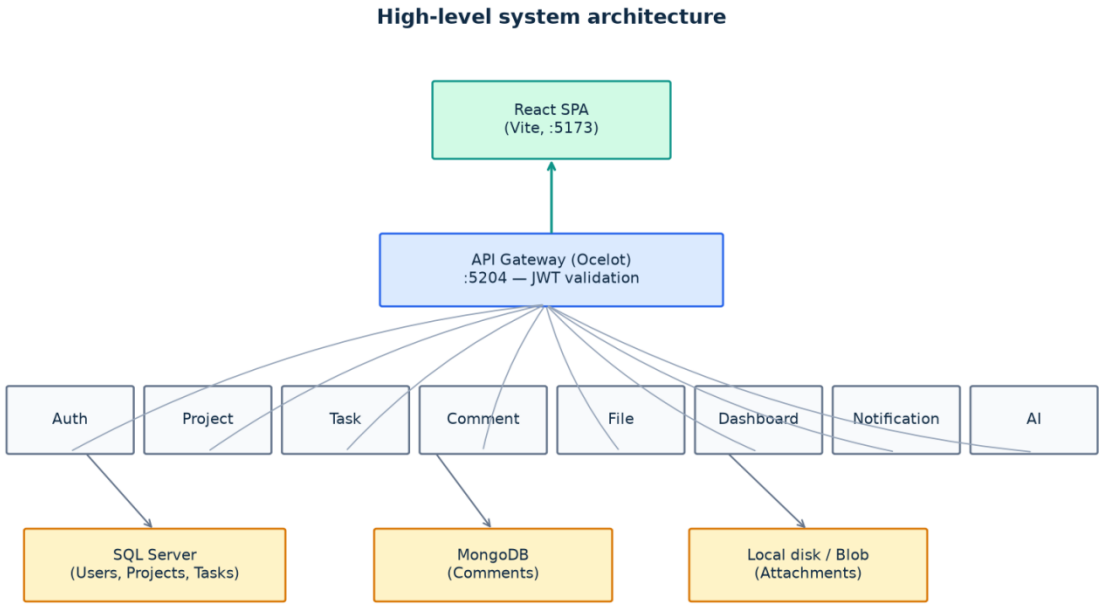


Figure 1 — Logical architecture (development topology)

3. Technology Stack and Implementation

Layer	Technologies	Notes
Frontend	React 19, Vite, Tailwind CSS, Axios	Kanban via @hello-pangea/dnd; forms with React Hook Form + Zod
API edge	ASP.NET Core + Ocelot	Single public HTTP entry; CORS aligned with Vite dev server
Services	.NET 10 Web APIs	EF Core on SQL where relational data fits; Mongo driver for comments
Real-time	SignalR	Notification bell updates without full page refresh
Local infra	Docker Compose	SQL Server + MongoDB; File service uses disk storage in Development
Quality	xUnit, Vitest, Playwright, GitHub Actions	CI builds backend and frontend on pull requests

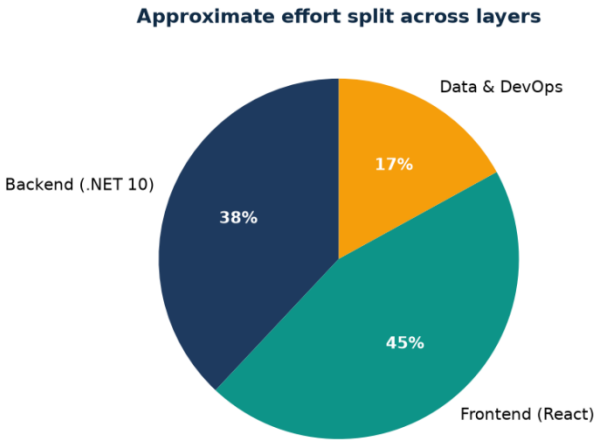


Figure 2 — Rough effort distribution (team estimate)

Entity Framework migrations run automatically in Development for Auth, Project, Task, and File, which kept onboarding friction low for new machines. Shared libraries centralise JWT validation helpers and current-user context so each service does not reimplement security primitives. One practical lesson we documented in the repository is assignee access: users who are only assigned to a task still need read access to that project's board. We solved it by merging task-based project IDs into the project access check and auto-adding assignees to the project team when an admin assigns work.

3.1 Demonstration dataset

For presentations we added an idempotent PowerShell seed script. It creates the Acme Digital team, three sample projects (including Customer Mobile App), sprints, seven tasks with subtasks, assignments, and comments. Reviewers can log in as admin@local.test or user@local.test using passwords from a local .env file—secrets never committed to source control.

4. Testing and Results

We treated automated tests as part of the deliverable, not an afterthought. Backend controllers and domain rules are covered with xUnit and in-memory or SQLite fixtures where appropriate. The gateway has configuration tests so a renamed route is caught before the UI silently fails. On the frontend, Vitest covers components and hooks; Playwright scenarios exist for end-to-end checks when both tiers are running.

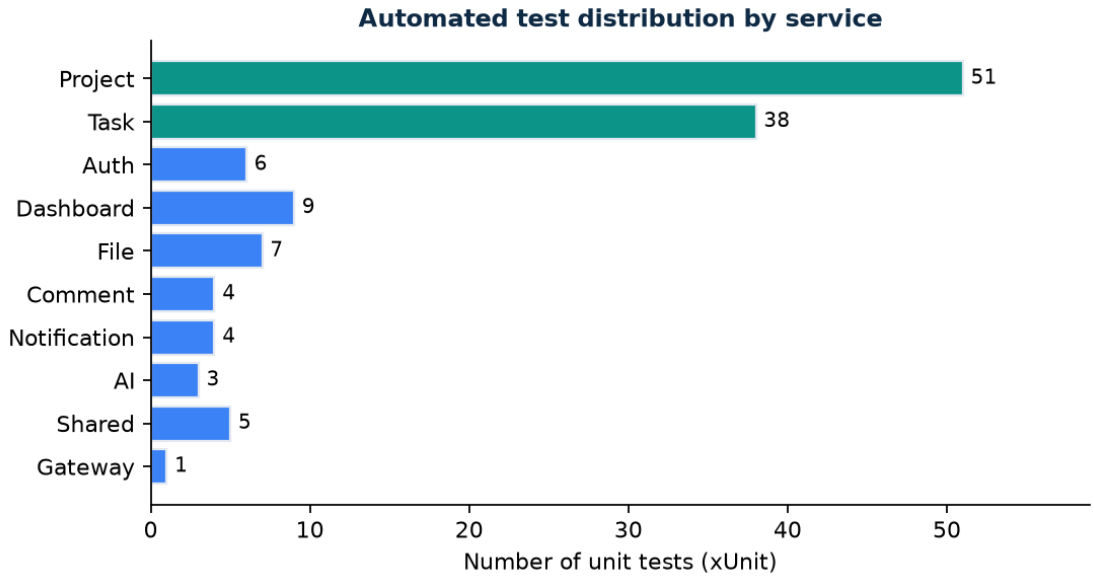


Figure 3 — 128 backend unit tests across ten test projects

Manual testing followed a short script: start Docker and services via start-local.ps1, seed demo data, log in as admin to create structure, then log in as a regular user to drag Kanban cards and attach a file. Health scripts probe each downstream port so we do not debug the UI when the Task service simply is not running. In class demonstration the flow—login, open Customer Mobile App, show sprint backlog, move a card to Done—completed in under three minutes.

5. Conclusion and Future Work

We met the original goal: a credible microservices sample that still feels like one product. The gateway pattern simplified the React client, and splitting comments into MongoDB avoided forcing every discussion thread into rigid SQL schemas. If we continued the project, the roadmap

already notes sensible next steps—structured email notifications, OpenTelemetry traces across services, and container deployment to Azure Container Apps or Kubernetes.

Personally, the most valuable outcome was learning to negotiate service boundaries as a team. Arguments about whether sprint logic belonged with tasks or projects were resolved by reading how each API was actually consumed in the UI. That discipline is transferable to industry work far beyond this classroom scope.

References

[1] Microsoft Learn — ASP.NET Core microservices architecture guidance.

[2] Ocelot documentation — API gateway configuration for .NET.

[3] Project repository — README.md, docs/LOCAL_DEVELOPMENT.md, docs/REPOSITORY_LAYOUT.md.

[4] React documentation — Vite tooling and modern SPA patterns.